

RP-02 — React Migration Strategy Decision: Group Lesson Planner

Metadata

Field	Value
Report id	RP-02
Date	2026-08-20
Subject	Choosing and phasing a migration of the Group Lesson Planner from one vanilla HTML file to React
Inputs consumed	docs/research/rp01-app-inventory/rp01-app-inventory.md (all 11 sections); index.html at commit 211eb6b (1473 lines); AGENTS.md ; docs/research/tools/package.json ; .gitignore ; 21 external sources listed in section 8
Verification statement	Behaviour facts are cited from RP-01 , which runtime-verified them; this report did not re-run the app. Code structure, line numbers, CSS rules and container-teardown claims in sections 4 and 6 were statically read from index.html in this worktree and each line number was re-checked. Repository contents were read from disk. All external claims were fetches from primary sources on 2026-08-20 and are quoted or paraphrased against the fetched text. Everything reasoned rather than read or fetched is labelled inference inline. No version numbers are pinned except the one documented Node stability index in section 8.

Executive summary

The correct strategy is **neither A nor B as stated, but a composite that takes A's technique and applies it at a different boundary, then uses B's sequencing**. Concretely: pin the **persisted data, the generated payment message and the CSV bytes** with executable tests, and *not* the DOM; then build a React app as a walking skeleton at a second path on the same origin over the same three `localStorage` keys, reach parity against the RP-01 §2 inventory, and cut over with a one-commit revert as the rollback. This is Strategy C.

Five findings drive that conclusion.

1. **RP-01 has already done the work characterization tests exist to do.** Characterization tests are defined as documenting a system's *actual* behaviour when no specification exists [S17][S19]. Here a runtime-verified specification does exist — RP-01 §2 (30+ features classified), §7 (exact observable strings) and §8 (26 defects). The marginal value of a broad DOM-level characterization suite over RP-01 is low. The marginal value of *executable* pinning is confined to the two outputs a human cannot eyeball: the 446-character payment message and the CSV export bytes.
2. **Holding a characterization suite green across the migration would lock in the defects.** RP-01 documents 26 defects including four Critical. Feathers' own stated purpose is "to document your system's actual behavior; not to check for the behavior you wish your system had" [S17]. Pinning D2,

D3, D6, D7, D12 and D19 as expected behaviour is the exact opposite of what this migration is for. Strategy A's premise — keep the tests green — is therefore wrong for a large slice of the surface.

3. **The DOM-level assertion surface is both hostile today and destroyed tomorrow.** RP-01 runtime-verified zero `data-testid` in the deployed file, three simultaneously-exposed buttons named `Cancel` and two named `Save`, modal open state not observable via `display` or `visibility`, and the `App` object reachable only because there is no bundler. Playwright's own guidance is that "Your DOM can easily change so having your tests depend on your DOM structure can lead to failing tests" [S14]. Every DOM assertion written before the migration is sunk cost by design — the refactoring literature says to delete such tests afterwards anyway [S18].
4. **The strangler-fig / island approach is not merely disproportionate here, it is structurally blocked.** The only three plausible React mount containers — `#groupList`, `#monthlyOverrides`, `#calendar` / `#calendar-dow` — are exactly the elements the vanilla renderer wipes with `innerHTML = ''` on every update (`index.html:1019`, `1062`, `1131–1132`). A React root mounted in any of them is destroyed by the next `render.groups()`. Making islands viable would require first rewriting the vanilla render layer to stop destroying its own containers, i.e. paying most of the migration cost in order to enable the incremental migration. There is also no routing, no module boundary and no second consumer to strangle.
5. **Most of the "DOM-entangled business logic" is not DOM-entangled.** `App.utils` (`index.html:1365–1467`) and `App.services.groups` / `App.services.csv` (`1214–1361`) are already DOM-free; they need *exports*, not extraction. The genuine entanglements are a short, nameable list of five items in section 6. The real structural defect is not DOM coupling at all: the schedule editor has a draft (`tempSelectedDates`, `tempMonthlyOverrides`) while the group-info form has none and writes straight to committed state — and that asymmetry is the direct cause of D2 and D7.

One correction to a shared assumption is recorded in section 7: a tracked `package.json` now exists at `docs/research/tools/package.json` (the report build pipeline), while the **application root still has none**. Any prompt invoking `npm run serve` or `npm run test:e2e` is citing a script that does not exist in any tracked file.

1. Objective and scope

In scope. Choosing between three migration strategies for moving the deployed application from one hand-written HTML file with an inline script to a React application, and producing a phased plan with entry and exit criteria. Deciding where automated tests should attach, and when DOM-entangled logic should be extracted.

Out of scope, and deliberately so. Choosing a state-management library; designing the component tree; the storage-durability question (a separate report); the accessibility remediation programme, beyond noting which parts the framework move makes free; purging the personal data from git history; and pinning tool versions.

Constraints taken as given.

Constraint	Source	Consequence for the decision
One deployed file, byte-identical to the committed <code>index.html</code>	RP-01 §1	Rollback is a single-commit revert and is total
GitHub Pages, static hosting, no backend	Programme contract	A build step requires either a committed build output or an Actions workflow [S20]
Served from a repository subpath	Live URL	Vite <code>base</code> must be set to <code>'/<REP0>/'</code> [S21]
Three <code>localStorage</code> keys, no schema version	RP-01 §4	The keys and persisted shape are the migration contract, not the DOM
Zero dependencies, zero build tooling today	RP-01 §5	The toolchain is introduced from zero; that cost belongs to whichever strategy is chosen
One non-technical end user, one part-time developer	Programme contract	Long unrewarded phases are an abandonment risk, not just a schedule risk
TypeScript preferred; Clean Code / Clean Architecture; "separate domain logic from UI concerns"	<code>AGENTS.md</code> : 18–21	Rules out Strategy B's terminal state (one <code>App</code> component holding everything)
No routing, no URL state, no network	RP-01 §5, §7	No route-level seam exists to strangle; but state is fully seedable from three keys

What is contract and what is not

This distinction decides everything else, and RP-01 supplies both halves.

Category	Items	Treatment
Contract — must survive byte-for-byte	The three key names and the persisted JSON shape (RP-01 §3-4), including <code>paymentTemplate</code> being a raw string and not JSON; the substituted payment message; the CSV header row, CRLF endings, all-fields-quoted convention and column order; the domain vocabulary and the <code>YYYY-MM</code> / <code>YYYY-MM-DD</code> key formats (RP-01 §6)	Pin with executable tests before touching anything
Not contract — free to change	All 47 element ids, <code>.group-card</code> , <code>.month-override-row</code> , <code>.day</code> , the <code>.show</code> visibility mechanism, the DOM shape, the event-wiring style, the CSS	Do not assert on any of it pre-migration
Wrong — must not be pinned	D2, D3 (subject to RP-01 open question 3), D6, D7, D12, D19, and the four Critical defects D0, D4, D5, D14	Fix during the move; record each as an intentional change
Vestigial — cheapest to drop now	<code>beforeunload</code> (<code>index.html</code> :541–546), <code>App.utils.formatDate</code> (1382–1387), <code>groupInfoForm.onSubmit</code> (492–495), <code>saveGroup</code> 's auto-save-schedule branch (673–675)	Drop, and record <code>beforeunload</code> as a deliberate user-visible change

Where each key question is answered

Question	Answered in
Q1 — score the three strategies on seven criteria	Section 2, with the evidence-strength column and the prose note beneath it
Q2 — characterization testing when selectors and markup will be destroyed	Section 4, subsection "Q2 answered"
Q3 — when test-first pays off and when it is wasted	Section 4, subsection "Q3 answered"
Q4 — how to evaluate strangler fig / islands at this size	Section 4, subsection "Why the strangler fig / island approach is rejected here"
Q5 — treatment of DOM-entangled business logic	Section 6
Q6 — the phases, with entry, exit, definition of done, risk and rollback	Section 5, table plus the per-phase definition-of-done prose

2. Strategy decision table

Strategy definitions used throughout:

- **A — test-first.** Write automated characterization/E2E tests against the current HTML app, then migrate and refactor while keeping those tests green.
- **B — scaffold-first.** Rough React init, move all existing logic into a single `App` component largely as-is, get it running, then add tests and refactor incrementally.
- **C — re-implement from the RP-01 specification behind contract pins.** Extract the already-pure logic to typed modules with unit tests; pin the persisted data, the payment message and the CSV bytes with approval tests; build a React walking skeleton at a second path on the same origin over the same three keys; reach parity against the §2 inventory; cut over. Islands/strangler-fig is evaluated and rejected as a variant of C in section 4.

Criterion	A test-first	B scaffold-first	C re-implement behind pins	Evidence
Silent behavioural regression	Mixed. Real cover for pre-migration edits, but a green suite pins 26 known defects as correct, and every DOM assertion dies at the first React commit	Worst. No executable pin on the payment message or CSV bytes at the moment of maximum change; those two are the only truly silent failures	Best where it matters. Pins sit on the storage shape, message text and CSV bytes, all of which survive by contract	strong

Criterion	A test-first	B scaffold-first	C re-implement behind pins	Evidence
Total effort to a working React app	Highest. 30+ features characterized before a line of React, then the DOM half rewritten	Lowest to first render, but the "as-is" move is impossible for the render layer, so the saving is smaller than it looks	Low-to-moderate. Phases 1–2 are small and are <i>kept</i> , not thrown away	moderate
Stable tests against legacy DOM	Poor. 0 <code>data-testid</code> deployed; 3× <code>Cancel</code> and 2× <code>Save</code> ambiguous; modal state invisible to <code>display/visibility</code> ; <code>App</code> backdoor dies under a bundler	Not applicable — tests are written later against a DOM the developer controls and can label	Good, by avoidance. No DOM assertions needed: state seeds from three keys, zero network, zero URL state	strong
Refactor velocity after milestone 1	Collapses. The green suite is the artefact the first React commit invalidates	High. The walking-skeleton effect: once a thin slice deploys, each addition is small	High , and the pins stay valid across every refactor because they sit on the contract	moderate
Rollback safety	Excellent before the move, weak at cutover (nothing pins the outputs)	Excellent <i>if</i> the old file stays deployed until cutover; B as stated does not say so	Excellent. Two clients over one storage contract; revert is one commit and total	strong
Live site working throughout	Yes, but delivers zero user value for a long stretch	Yes if cutover is deferred; the plan as stated is silent on it	Yes, and the new app is exercisable against the user's real data before cutover	strong
Suitability for one part-time developer	Poor. Long unrewarded phase; highest abandonment risk	Good early, but terminates in a God component that <code>AGENTS.md:19</code> forbids, so the debt is deferred not avoided	Good. B's sequencing without B's architectural terminus	weak

Reading the evidence column. *Strong* rows rest on RP-01's runtime verification plus a fetched primary source. *Moderate* rows reason from the code structure and from a documented practitioner account [S12], with no measurement for this app. The single *weak* row is a judgement about a person's motivation and available evening hours; it is stated because it is decision-relevant, not because it is measured. No effort figure in hours appears anywhere in this report — none is verifiable, and the contract forbids estimating.

Why B's central mechanic does not work as written. "Move all existing logic into a single `App` component largely as-is" is achievable for `App.utils` and `App.services` — roughly 250 of the ~1,110 script lines — and impossible for `App.render` (978–1183) and the 32 `on*` property assignments RP-01

counted. Those are imperative `innerHTML` builders and one-handler-per-event assignments; carrying them into React means either `dangerouslySetInnerHTML` (retaining D1, the stored XSS) or rewriting them as JSX. So B's "as-is" saving applies precisely to the code that should become typed modules anyway, and does not apply to the code that actually costs the effort. That is not a strawman reading; it is what the code shape forces. *Inference*.

3. Recommendation and the conditions under which it holds

Recommendation: Strategy C. Extract, pin the contract, build a walking skeleton beside the live app on the same origin, reach parity against the RP-01 inventory, cut over with a one-commit rollback. Phases are in section 5.

C is deliberately not a rejection of A and B. It is **A's technique relocated and B's sequence retained**:

- From A, keep executable pinning — but at the storage, message and CSV boundary, which RP-01 §7 verified is fully seedable and fully observable without a single DOM selector.
- From B, keep the walking skeleton — the practice associated with Alistair Cockburn of building the thinnest end-to-end slice that actually builds, deploys and runs, so that architecture and functionality grow together. Deploy it early, at its own path. This report cites the practice by name only; no fetchable primary source for it was obtained (see section 7).
- From B, reject only the terminus: no single `App` component, because `AGENTS.md:19` requires domain logic separated from UI concerns and because the extraction in section 6 is cheaper done first than undone later.

Answering the head-on objection: this is a rewrite, and rewrites are the classically-warned-against option

Fowler's case against replacement is specific, not general. He gives three mechanisms: "Replacing a serious IT system takes a long time, and the users can't wait for new features. Replacements seem easy to specify, but often it's hard to figure out the details of existing behavior" [S4]. Each mechanism is separately neutralised here, and each neutralisation is a **condition** on the recommendation — if one stops holding, section 4 applies.

Failure mechanism for rewrites	Why it does not bite here	Verified by
Existing behaviour is hard to specify	RP-01 §2 enumerates 30+ features with a core/incidental/vestigial classification; §7 records the exact user-visible strings; §3 records the exact persisted shape	RP-01, runtime-verified
Takes too long; users cannot wait	1,473 lines total, of which ~75 percent is the script, zero dependencies to port, one user with no feature backlog	RP-01 §5
Big-bang deployment risk	The old app stays deployed at its own path throughout; cutover is one commit and reverts to a byte-identical file	RP-01 §1

Fowler's condition for legitimately discarding code is also met: "the team that writes the sacrificial architecture is the team that decides it's time to sacrifice it" [S5]. Here the author of the file is the person deciding — the opposite of the pattern Fowler warns about, where a new team dismisses inherited code it does not understand.

The one genuinely incremental element that is worth keeping

Islands are blocked (section 4), but branch-by-abstraction is not — it just applies at a different layer. "Use an abstraction layer to allow multiple implementations to co-exist in the software system. Use the notion of one abstraction and multiple implementations to perform the migration" [S6]. **The three `localStorage` keys already are that abstraction.** Two complete clients — the vanilla file at the root and the React build at a second path — can coexist over one persisted contract on one origin, with the user able to switch between them and each reading the other's writes. That gives the strangler fig's real benefit (gradual, reversible, always-working) without needing any DOM seam. It also means the new app can be validated against the teacher's actual data before cutover, which no DOM-level test suite could achieve.

The cost is stated plainly: while both clients exist, the React app must be **read-compatible and write-identical**. Any shape change is deferred to after cutover. Phase 3's exit criterion enforces this.

Conditions under which the recommendation holds

1. RP-01 §2 is complete — no feature the teacher relies on is missing from it.
2. The developer accepts that the defects in section 1's "wrong" row are to be fixed, not preserved.
3. The GitHub Pages publishing source can serve a built artifact from a second path (see section 7 TBD).
4. No new feature work is requested by the end user mid-migration.
5. The migration is done by one person who can hold the inventory as a checklist.
6. `paymentTemplate` remains a raw string and the two JSON keys keep their exact shape until cutover.

4. Conditions that would flip the recommendation

#	Condition	Flips to	Detection
F1	RP-01 §2 proves incomplete — the teacher demonstrates a behaviour not in the inventory	A, at the data boundary. The "spec already exists" premise is what makes C cheap	Walk the app with the teacher before Phase 4; compare against §2 row by row
F2	The developer decides current behaviour is the requirement — e.g. D3 cross-month price bleed is intended (RP-01 open question 3)	A for the money paths specifically; characterization value rises sharply where behaviour must be preserved exactly	Settle open question 3 before Phase 2
F3	Pages cannot serve a built artifact (Actions unavailable, or branch-only publishing with no usable subfolder)	B with a single hard cutover, plus a committed build output; the two-clients phasing is lost	Open repository Settings then Pages

#	Condition	Flips to	Detection
F4	New features are requested during the migration	Islands become worth their cost despite the container teardown, or Phase 4 is hard time-boxed and the feature ships in the vanilla file first	Any feature request from the teacher
F5	A second developer joins	A gains value: an executable spec beats a checklist held in one head	Team change
F6	The app grows materially — routing appears, or the script grows several-fold	The sacrificial-architecture argument [S5] weakens; incremental displacement patterns [S10] start to apply	Line count and the appearance of URL state
F7	The diverged <code>index.html</code> lands and its <code>data-testid</code> names are deliberately reproduced in the React build	A's DOM hooks stop being sunk cost, and a subset of pre-migration DOM tests can survive the move. This is the only condition under which they do	Decide RP-01 open question 1, then decide whether to carry the names forward

Why the strangler fig / island approach is rejected here — Q4 answered directly

Incrementality's overhead is justified by three documented conditions. None holds.

Condition that justifies incrementality	Documented basis	Status here
A full replacement cannot be delivered before users need features	"the users can't wait for new features" [S4]	Fails. One user, no backlog, 1,473 lines
Big-bang deployment risk is unacceptable at the user volume	"Suggestions of alpha and beta user groups were considered unacceptable given the huge volumes of users" [S11]	Fails. One user; cutover reverts with one commit
A module, route or capability seam already exists to strangle	"various parts of the software system are dependent on a module, library, or framework that we wish to replace" [S6]; seam identification [S10]	Fails. One global <code>App</code> object, zero imports, zero exports, zero routing (RP-01 §5)

And a fourth, decisive, structural objection specific to this file. React's own documented island path requires you to "find that HTML element with `document.getElementById` and pass it to `createRoot`" [S1]. The only sensible targets are `#groupList`, `#monthlyOverrides`, `#calendar` and `#calendar-dow`. Statically read in this worktree:

Container	Wiped at	Wiped by
<code>#groupList</code>	<code>index.html:1019</code>	<code>render.groups()</code> , on every group change
<code>#monthlyOverrides</code>	<code>index.html:1062</code>	<code>render.monthlyOverrides()</code> , on every month/price change
<code>#calendar</code>	<code>index.html:1131</code>	<code>render.calendar()</code> , on every date toggle

Container	Wiped at	Wiped by
#calendar-dow	index.html:1132	render.calendar() , on every date toggle

A React root mounted in any of these is destroyed by the next vanilla render. Note that RP-01 §5 records the same destroy-and-rebuild fact and draws the correct conclusion for *behaviour* — "semantically identical to React re-render, so behaviour will not change" — but the same fact has the opposite sign for *incrementality*. Islands here would require first rewriting `App.render` to stop destroying its own containers, which is most of the migration. That is a far stronger argument than "the app is small", and it is why C's viable form is clean re-implementation rather than islands.

A last honesty note on the case-study literature. Every documented legacy-to-React migration this report could fetch operates at a scale where incrementality is forced: millions of users behind a native shell [S11], a business-critical ticketing platform shipping multiple times a week [S12], and a patterns catalogue explicitly framed for "large organizational contexts with significant complexity" [S10]. Their conclusions transfer as **rationale, not as recipe**. Copying their phasing into a 1,473-line single-user tool is exactly the disproportionate advice the programme contract asks be rejected.

Q2 answered: characterization testing when the selectors and markup will be destroyed

Selector stability. Playwright states both halves of the tension and they are reconcilable: "Testing by test ids is the most resilient way of testing as even if your text or role of the attribute changes, the test will still pass. However testing by test ids is not user facing" [S13], and separately, "Your DOM can easily change so having your tests depend on your DOM structure can lead to failing tests" [S14]. Resolution for this app:

Assertion target	Survives the migration?	Verdict
CSS class / id selectors	No — all 47 ids and every class are free to change	Never assert
data-testid	Only if the React build reproduces the same names deliberately	Conditional; see F7
Role + accessible name	Not reliable today. RP-01 verified three buttons named <code>Cancel</code> and two named <code>Save</code> simultaneously exposed, because closed modals stay in the tree	Becomes reliable only after the modal-visibility fix. Then it is the best DOM-level target
Visible text strings	Yes for the RP-01 §7 strings, which are contract-adjacent, but three of them are being deliberately changed (D19 pluralisation)	Assert only after the wording is settled
The three <code>localStorage</code> keys and their shape	Yes, by contract	Assert here
The generated payment message	Yes, by contract	Assert here , with approval tests

Assertion target	Survives the migration?	Verdict
The CSV export bytes	Yes, by contract	Assert here , with approval tests

The sunk cost is not an accident — the literature treats it as the design. Test Double's refactoring guide instructs: "Delete the characterization tests. They should be redundant if tests of the newly refactored code are complete, and redundant coverage is problematic" [S18]. So the honest framing of Strategy A's cost is not "you might have to rewrite the tests" but "you *will* delete them, by design; is the safety net worth the throwaway cost?" For DOM-level tests against this app the answer is no, because RP-01 already provides the documentation those tests would produce, and because the assertion surface is hostile before it is even destroyed. For the three contract targets the answer is yes, because those tests are *not* throwaway — they outlive the migration unchanged.

Assertions should target user-visible behaviour and persisted data rather than DOM structure. That is Testing Library's stated first principle — "The more your tests resemble the way your software is used, the more confidence they can give you" [S15] — and Playwright's: tests should "avoid relying on implementation details such as things which users will not typically use, see, or even know about such as the name of a function, whether something is an array, or the CSS class of some element" [S14]. For this app the strongest such surface is not the DOM at all. RP-01 §7 verified that state can be seeded entirely by writing three keys before load, with zero network, zero URL state and zero cookies. That single property makes a durable contract test cheap **today**, with no `data-testid` and no browser at all for the pure paths.

Q3 answered: when test-first pays off, and which conditions apply here

#	Documented condition for test-first paying off	Basis	Applies here?
1	Current behaviour is the requirement	Characterization tests "document your system's actual behavior; not... the behavior you wish your system had" [S17]	No. 26 defects, four Critical; six behaviour changes already queued in RP-01 §11
2	The change is behaviour-preserving refactoring	Refactoring is defined as change "without changing its observable behavior" [S7]	No. This is replacement, not refactoring, by that definition
3	The code under test survives the change	Implicit in [S18]'s delete-afterwards rule	Partly. True for <code>App.utils</code> and <code>App.services</code> ; false for <code>App.render</code> and all event wiring
4	Assertions can attach to a boundary that survives	[S6] abstraction/implementation split; [S14] avoid DOM structure	Yes , but only at storage / message / CSV. Not at the DOM
5	No usable specification of current behaviour exists	The purpose of characterization testing [S17] [S19]	No — decisive. RP-01 is that specification, runtime-verified

#	Documented condition for test-first paying off	Basis	Applies here?
6	The feedback loop is cheap: seedable state, no auth, no network	[S15] guiding principle; RP-01 §7 isolation properties	Yes , and unusually so
7	Confidence is the blocker on making changes at all	"Old codebases are often terrifying places, where developers fear to change working code" [S8]	Partly. True for the money and message paths; false for the presentational layer
8	A seam can be introduced at acceptable cost	"A team can spend several months figuring out how to introduce seams into a well-worn legacy system" [S9]	Yes, cheaply — but only because the seam is the storage layer, which already exists

Conditions 4, 6 and 8 hold; 1, 2 and 5 fail. That is the whole answer: **test-first is right about the technique and wrong about the target**. The conditions that survive all point at the same three artefacts — persisted data, generated message, CSV bytes — and every failing condition points away from the DOM.

Where test-first is documented as wasted effort: when the tests will be deleted and their coverage replaced [S18]; when they encode behaviour that is known to be wrong and will therefore be "approved" into permanence [S17]; and when the specification they would produce already exists. All three apply to a DOM-level suite here.

5. Phased plan

Definition of done for each phase is stated in prose after the table, because the table is already at the seven-column maximum.

Phase	Goal	Tasks	Entry criteria	Exit criteria	Risk	Rollback
P0 Freeze the baseline	Settle what "current behaviour" means	Decide RP-01 open question 1; confirm Pages publishing source; land the wording and privacy fixes	RP-01 accepted	Exactly one <code>index.html</code> committed and deployed; publishing source recorded; no uncommitted app diff	The diverged copy's testids tempt DOM pinning that cannot survive	None needed; no architecture changed

Phase	Goal	Tasks	Entry criteria	Exit criteria	Risk	Rollback
P1 Extract the pure domain	Typed, unit-tested modules, framework-free	Create root <code>package.json</code> and <code>tsconfig.json</code> ; port <code>App.utils</code> and <code>App.services</code> . groups / csv as pure functions; type the currency union	P0 exit met	Every "already DOM-free" item in section 6 is a typed export with tests; <code>node --test green</code> ; <code>index.html</code> untouched	Copy and live file diverge while both exist	Delete the new directories; nothing deployed depended on them
P2 Pin the contract	Executable pins on the three surviving outputs	Approval tests for the substituted message and CSV bytes; CSV round-trip; storage read/write shape test	P1 exit met	Approved files committed; each labelled contract or defect	Approving buggy output as correct [S17]	Delete the tests
P3 Walking skeleton	A deployed React app beside the live one, same origin, same keys	Vite + React + TS with <code>base</code> set to the repo subpath [S21]; storage adapter reusing P1 modules; thinnest slice: list, add, persist, reload	P1-P2 exit met; publishing source resolved	React app live at its own path, renders the user's real data, one add-persist-reload cycle passes; root <code>index.html</code> unchanged	Build step vs publishing source mismatch; wrong <code>base</code> yielding asset 404s	Remove the second path; root file untouched, live app unaffected

Phase	Goal	Tasks	Entry criteria	Exit criteria	Risk	Rollback
P4 Parity by inventory	Implement §2, drop the vestigial, fix the defects	Work RP-01 §2 as a checklist; route money and message through P1 modules; make the edit draft the only write target; <code>hidden</code> -based modals, roles, focus, keyboard cells	P3 exit met	Every §2 row implemented or listed as deliberately dropped with a reason; every RP-01 defect fixed or deferred with a reason	Scope creep into a11y/i18n; silent parity drift because the checklist is manual	Stop at any point; the live app is a different path
P5 Cutover	React build becomes the root artifact	Publish the build to root; move the vanilla file to a stable fallback path; verify against real data before announcing	P4 exit met; pins green; user has taken a CSV export	Root URL serves the React build; fallback path serves the old file; both read the same keys	First-load failure with no fallback the user knows about	Revert one commit; restores a byte-identical single file
P6 Retire scaffolding	One app, one toolchain	Delete the fallback after an agreed period; delete pins now duplicated by unit tests [S18]; delete the dead CSS; correct <code>AGENTS.md</code>	Agreed observation period after P5 with no rollback	No duplicated coverage; documented commands all exist and pass	Removing the fallback before the user has a backup habit	Git history

Definition of done, per phase

- **P0.** `git status` shows no modification to `index.html` in any checkout; the deployed bytes match the committed bytes (the check RP-01 §1 already performed); the publishing source is written down somewhere durable; the four prior-art `docs/*.md` files either match the committed file's line numbers or are corrected. If the diverged copy is discarded rather than landed, the correction of those line numbers is mandatory, not optional — RP-01 verified they are off by 8-18 lines.
- **P1.** A single command runs the unit tests and it is the same command `AGENTS.md` documents. Node's built-in test runner is documented Stable and needs no test-framework dependency [S22], which suits the zero-budget constraint; that choice is a preference, not a requirement of this plan.

`index.html` is byte-identical to its P0 state.

- **P2.** Each approved file carries, in the test that produces it, an explicit label saying whether it encodes contract (must never change) or defect (expected to change at P4). Without that label the suite silently becomes a defect-preservation harness, which is the documented failure mode [S17].
- **P3.** The vanilla app and the React app can be opened alternately in the same browser profile and each reads data the other wrote. This is the concrete test of the branch-by-abstraction claim in section 3.
- **P4.** The parity checklist is committed and every RP-01 §2 row has a disposition: implemented, deliberately dropped, or deferred. `beforeunload` is the worked example — see below.
- **P5.** The user has been shown the fallback URL, and has taken a CSV export **before** cutover, not after. Given RP-01's D16 (export omits the template) and D17 (no BOM), the export is a partial backup; say so when asking for it.
- **P6.** `AGENTS.md` describes only commands that exist. RP-01 verified it currently documents four npm scripts that no tracked file provides.

Dropping vestigial features is a deliberate change, not a regression

The migration is the cheapest moment to delete dead code, and RP-01 §2 already names it:

`App.utils.formatDate` (`index.html:1382–1387` , never called), `groupInfoForm.onSubmit` (`492–495` , proven unreachable), `saveGroup` 's auto-save-schedule branch (`673–675` , proven unreachable), and about 35 lines of dead CSS. Deleting those is invisible to the user and needs no announcement.

`beforeunload` (`index.html:541–546`) is different, and it is the worked case. RP-01 classifies it vestigial because browsers discard its custom text, so its CSV-backup instruction is never seen. But the *prompt itself* is observable: it fires on every navigation once any group exists. Removing it is therefore a **deliberate, user-visible behavioural change** — the teacher will notice that the "leave site?" dialog stopped appearing — and it must be recorded as such in the P4 checklist and mentioned to her, not filed under dead-code removal. RP-01 open question 6 asks whether it is wanted at all; that question must be answered rather than resolved by omission. The same discipline applies to every D-numbered fix: a fixed defect changes behaviour, and behaviour changes get recorded.

6. Logic-extraction plan for DOM-entangled business logic

The brief's premise is partly wrong and saying so is the finding: **most of the business logic is not entangled with DOM code**. Statically read in this worktree, the whole of `App.utils` (`index.html:1365–1467`) and the whole of `App.services.groups` and `App.services.csv` (`1214–1361`) touch no DOM at all. They need *exports*, not extraction. Naming the five real entanglements precisely is worth more than a blanket "extract everything first".

#	Logic	Location	Coupled to	When to extract
E1	Date/month keys, padding, weekday maths, CSV escape and parse, month-key normalisation, deep clone	1382–1466	Nothing	Before. Pure already; copy into typed modules

#	Logic	Location	Coupled to	When to extract
E2	CSV serialise and deserialise; override create/normalise/clone; dates-by-month	1214–1361	Nothing	Before. Pure already
E3	Currency formatting	1379–1381	Nothing, but the whitelist lives in <code><option></code> markup at 292–293 and is read back at 1001–1002	Before. Move the whitelist to a typed union; this is a business rule stored in markup and the source of D5's split display
E4	Payment-message templating	1367–1378	Not the DOM — <code>localStorage</code> via <code>getTemplate()</code> (1207–1209) and the <code>App.state.isEditing</code> global via <code>getActiveOverrides()</code> (1242–1245)	Before. Make template and overrides parameters. Storage/global coupling is much cheaper to break than DOM coupling
E5	Month total <code>lessonCount * monthData.price</code> , and <code>selectedInMonthCount</code> accumulated as a side effect of the render loop	1086, 1113, and 1148–1159 feeding 1169	Genuinely DOM: computed inside <code>innerHTML</code> builders and a render loop	Before for the two pure selectors (<code>monthTotal</code> , <code>datesInMonth</code>); during for the mutation and event wiring, which React replaces
E6	<code>saveGroup</code> branching on <code>calendarContainer.style.display === 'block'</code>	673	Reads presentation to decide logic	Delete, do not extract. RP-01 runtime-verified the branch is unreachable
E7	Temp-vs-committed override switching	<code>state</code> 366–372, clone 1409–1411, read 1242–1245, commit 745–772	Global mutable state, not DOM	During. This is state design, and React gives it a natural home

E7 in detail — the finding that matters most

RP-01 §5 says the temp-copy pattern "is sound and worth keeping", and that is right as far as it goes. Read across the handlers, however, the pattern is **applied inconsistently, and that inconsistency is the mechanism behind D2 and D7**:

- The **schedule editor has a draft**. `startEditingDates` (718–743) seeds `tempSelectedDates` and `tempMonthlyOverrides`; `Done` → `saveDateChanges` (745–772) commits and persists; `Cancel` → `cancelDateChanges` (774–781) discards. Correct copy-on-write.

- The **group-info form has no draft at all**. `updateDefaultPrice` (690–705) writes `group.price` and mutates `group.monthlyOverrides[...]` on **committed** state immediately, without persisting. `cancelGroupInfoEdit` then re-reads state that has already been mutated — that is D2. And `updateDefaultPrice` finishes by calling `render.groupInfo()` (704), which unconditionally rewrites all three inputs from saved state (1008–1010), discarding an unsaved name edit — that is D7.

A precise correction to RP-01 §4 is warranted here. RP-01 groups `updateDefaultPrice` and `handleSelectedDatesPriceChange` together as "two mutation paths [that] never persist... both of which mutate live state". They are structurally different. `handleSelectedDatesPriceChange` (858–881) writes through `getActiveOverrides` (1242–1245), which returns `tempMonthlyOverrides` whenever `App.state.isEditing` is true; and its input `#selectedDatesPriceInput` (332) sits inside `#calendar-container` (316), which is `display: none` until `startEditingDates` sets it to `block` (739) and sets `isEditing = true` (722). So it mutates the **draft**, and not persisting is *correct* there because `Done` persists. Only `updateDefaultPrice` writes to committed state without saving. The distinction matters because it localises D2 and D7 to a single missing draft rather than to a general persistence problem.

Consequence for the design. In React, making the edit draft the single write target for the whole group-editing session — info fields included — removes D2 and D7 structurally rather than by patching two handlers. Two constraints, both from RP-01 §5: a `Set` must not leak into the serialised store (`tempSelectedDates` is a `Set`), and the `JSON.parse(JSON.stringify(...))` clone at 1409–1411 should be replaced by something typed rather than carried forward.

Sequencing rationale

Extract E1-E4 **before** any framework work, for three reasons that do not depend on React at all: the functions are already pure, so the extraction is mechanical and low-risk; they become unit-testable for the first time, which is where the money arithmetic and the message text live; and they are the artefacts P2's pins assert against, so the pins can be written without a browser. RP-01 §5 reaches the same conclusion independently — "Extracting `services` and `utils` first gives pure functions with no DOM coupling — the cheapest early win in a React migration".

Do the extraction **by copy**, into new files, leaving `index.html` untouched until cutover. Duplication here is deliberate and temporary: the copy is the migration target, and the deployed file's zero-dependency, zero-build property is preserved until P5. Modifying `index.html` to import modules would force `type="module"`, changing script timing and strictness, for no benefit before P3.

Defer E5's event wiring, E7's state redesign and all of `App.render` to **during** the move. Extracting a DOM builder before replacing it produces an abstraction shaped by the thing being deleted. Branch-by-abstraction's own framing supports this: the abstraction layer is scaffolding, and "We may also choose to delete the abstraction layer once we no longer need it for migration" [S6]. A layer that exists only to be deleted should not be built where the deletion is already certain.

7. Risks, contradictions and unknowns

Corrections and additions to the inputs

Item	Input claim	Finding
Root package.json	RP-01 §1: "No README, no package.json, no .github/, no CI config"	Accurate for the application , and still is. But <code>git ls-files</code> in this worktree now shows tracked <code>docs/research/tools/package.json</code> and <code>package-lock.json</code> — the report build pipeline, whose only script is <code>build → node build-report.js</code> . Any downstream prompt invoking <code>npm run serve</code> , <code>npm run test:e2e</code> , <code>npm run test:ui</code> , <code>npm run test:update</code> or <code>npm run test:trace</code> names a script that exists in no tracked file. <code>AGENTS.md:10–15</code> documents all five
The two never-persisting mutation paths	RP-01 §4 groups <code>updateDefaultPrice</code> and <code>handleSelectedDatesPriceChange</code> as both mutating live state without persisting	Half right. <code>handleSelectedDatesPriceChange</code> mutates the draft via <code>getActiveOverrides</code> and is only reachable while <code>isEditing</code> is true; not persisting is correct there. Only <code>updateDefaultPrice</code> writes committed state without saving. See section 6
Destroy-and-rebuild rendering	RP-01 §5 draws the correct behavioural conclusion: no regression risk from React re-render	Addition, not a correction: the same fact blocks the island migration , because the only mount targets are the wiped containers (<code>1019</code> , <code>1062</code> , <code>1131–1132</code>). RP-01 did not note this consequence
The diverged copy's <code>data-testid</code> hooks	RP-01 §1 presents them as valuable prior art that changes "the entire test-anchor surface"	True, and valuable for verifying pre-migration bug fixes. But near-worthless as migration-spanning anchors, since DOM hooks are exactly what the migration destroys — unless F7 holds

Unresolved items

Item	Value	Why unresolved	What resolves it
GitHub Pages publishing source, branch and folder	TBD	Repository Settings are not publicly readable; inherited from RP-01	Open Settings then Pages on the repository. Load-bearing : it decides whether P3's two-path coexistence is possible at all, and it is the trigger for F3
Whether the diverged main-checkout <code>index.html</code> lands	TBD	Developer decision; inherited from RP-01 open question 1	The P0 decision gate. Both branches are specified in section 5

Item	Value	Why unresolved	What resolves it
Whether D3 cross-month price bleed is intended	TBD	RP-01 open question 3; the developer's own prior-art doc asks the same question	A product decision, needed before P2 labels it contract or defect. Trigger for F2
Whether the teacher uses the app on a phone	TBD	Not observable from the artifacts	Ask her. Decides the priority of the 375 px calendar-control fix and how much P4 owes to responsive layout
React, Vite, Playwright and Node versions to pin	Deliberately not stated	Version numbers cannot be verified without fetching a registry, and the contract forbids estimating	Resolve at install time and record in the committed lockfile. This report's plan does not depend on any specific version
Primary text for the walking-skeleton practice	TBD	https://alistair.cockburn.us/walking-skeleton/ returned HTTP 404 on 2026-08-20; the O'Reilly chapter reproducing the definition is subscription-gated	Cockburn's <i>Crystal Clear</i> , or a reachable replacement for the 404'd page. Nothing in this report's argument depends on it; it is named, not quoted

Contradictions between sources, stated rather than resolved

- 1. react.dev recommends a framework; this app should not use one.** "If you want to build a new app or website with React, we recommend starting with a framework" [S2]. The same documentation permits the alternative and states its conditions: build from scratch if "you are confident you will never need features like SSR, SSG, or RSC" and are "comfortable tackling framework-like problems on your own" [S3]. For a single-user, no-backend, no-routing, static-hosted tool the second condition set is satisfied and the first recommendation is disproportionate. Recording the disagreement rather than hiding it: react.dev's own primary recommendation points the other way, and this report knowingly declines it on the stated conditions.
- 2. Playwright says test-ids are the most resilient thing to test, and also says not to test implementation details.** Both quotes are in section 4's Q2 answer, both from [S13] and [S14]. They are not actually contradictory — resilience-to-markup and fidelity-to-user are different axes — but a reader who takes either sentence alone will reach a different plan. The conditions under which each holds are in the Q2 table.
- 3. The migration literature is near-unanimously pro-incremental, and near-unanimously drawn from large systems.** [S4], [S10], [S11] and [S12] all favour gradual replacement; all four are set in contexts where a big bang was not survivable. [S10] contains no threshold below which incrementality stops paying, and explicitly does not discuss small systems. [S5] supplies the counterweight but on a different axis (scale growth, not size). **No fetched source states a size threshold.** This report's rejection of islands therefore rests on the structural container-teardown evidence in section 4, not on a citable size rule — that distinction is deliberate.
- 4. Feathers' technique versus Feathers' purpose.** Characterization tests are the documented way to get untested legacy code under test [S9][S19], and their stated purpose is to document actual, not desired, behaviour [S17]. When the goal is to change a documented-wrong behaviour, the technique's

own purpose statement excludes the use. This is a genuine tension inside the source material, not a disagreement between sources, and it is why section 4 concludes that A is right about technique and wrong about target.

Standing risks in the recommended plan

Risk	Mechanism	Mitigation
Data corruption while two clients share three keys	The React app writes a shape the vanilla app cannot read, before cutover	P3 exit criterion: read-compatible and write-identical until P5. Defer every shape change to after cutover. <code>paymentTemplate</code> stays a raw string
Personal data leaking into a committed approved file	The default template at <code>index.html:382-400</code> contains a real name, IBAN and tax identifier (RP-01 D0); an approval test that uses the default template would commit them to a new file	P2 must use a neutral test template. This is stated as a hard constraint in the P2 quick-win prompt below
Silent parity drift	The P4 checklist is manual; a missed RP-01 §2 row is exactly the silent regression the plan exists to prevent	Commit the checklist with a disposition per row; make "every row has a disposition" the exit criterion, not "it looks done"
Approving defects into permanence	The documented failure mode of approval testing: snapshots updated without review [S17]	Label every approved file contract or defect at P2
Abandonment	A single part-time developer stalling mid-migration leaves two half-apps	P3's second path means every stall point is a working live site plus an unfinished experiment, never a broken deployment
The fallback is removed too early	The user loses her escape hatch before trusting the new app	P6 entry criterion is an agreed observation period, not a date

8. Sources

#	Title	URL	Accessed	Supports
S1	React — Add React to an Existing Project	https://react.dev/learn/add-react-to-an-existing-project	2026-08-20	The documented island path: find the element with <code>getElementById</code> and pass it to <code>createRoot</code> ; Vite named as the JS environment; "start with small interactive components... gradually keep moving upwards"

#	Title	URL	Accessed	Supports
S2	React — Creating a React App	https://react.dev/learn/creating-a-react-app	2026-08-20	react.dev's primary recommendation is to start with a framework; frameworks support CSR/SPA/SSG deployable to static hosting without a server
S3	React — Build a React App from Scratch	https://react.dev/learn/build-a-react-app-from-scratch	2026-08-20	Vite, Parcel and Rsbuild as the documented from-scratch build tools, and the stated conditions for choosing this path over a framework
S4	Martin Fowler — Strangler Fig Application	https://martinfowler.com/bliki/StranglerFigApplication.html	2026-08-20	The three mechanisms by which replacements fail, and the claim that reduced risk outweighs incremental cost
S5	Martin Fowler — Sacrificial Architecture	https://martinfowler.com/bliki/SacrificialArchitecture.html	2026-08-20	When discarding code is legitimate, and the condition that the team who wrote it decides
S6	Martin Fowler — Branch By Abstraction	https://martinfowler.com/bliki/BranchByAbstraction.html	2026-08-20	One abstraction with multiple coexisting implementations; system must build and run at all times; the layer is temporary scaffolding
S7	Martin Fowler — Definition of Refactoring	https://martinfowler.com/bliki/DefinitionOfRefactoring.html	2026-08-20	Refactoring is change "without changing its observable behavior", which excludes this migration from the definition
S8	Martin Fowler — Self Testing Code	https://martinfowler.com/bliki/SelfTestingCode.html	2026-08-20	Confidence as the thing tests buy, and the fear that untested legacy code induces
S9	Martin Fowler — Legacy Seam	https://martinfowler.com/bliki/LegacySeam.html	2026-08-20	Feathers' seam definition, and the documented cost of introducing seams into well-worn legacy code
S10	Martin Fowler — Patterns of Legacy Displacement	https://martinfowler.com/articles/patterns-legacy-displacement/	2026-08-20	Incremental displacement patterns; risk mitigation must be explicitly valued; the catalogue's framing is large organisational contexts
S11	Thoughtworks / Martin Fowler — Using the Strangler Fig with Mobile Apps	https://martinfowler.com/articles/strangler-fig-mobile-apps.html	2026-08-20	Documented case study: user volume made a big bang unacceptable, and the pattern's temporary overhead was accepted
S12	Jack Franklin — Migrating complex JavaScript applications	https://www.jackfranklin.co.uk/blog/migrating-complex-javascript-angular-react/	2026-08-20	Practitioner account of an incremental Angular-to-React migration, and the value of tests decoupled from which framework renders the UI

#	Title	URL	Accessed	Supports
S13	Playwright — Locators	https://playwright.dev/docs/locators	2026-08-20	Test ids are the most resilient target but are not user-facing; role locators are the recommended priority
S14	Playwright — Best Practices	https://playwright.dev/docs/best-practices	2026-08-20	Test user-visible behaviour, avoid implementation details, and DOM-structure coupling causes failing tests
S15	Testing Library — Guiding Principles	https://testing-library.com/docs/guiding-principles/	2026-08-20	"The more your tests resemble the way your software is used, the more confidence they can give you"; discouraging tests of implementation details
S16	ApprovalTests	https://approvaltests.com/	2026-08-20	Approval / golden-master technique: compare whole outputs against an approved snapshot instead of writing per-field assertions
S17	Understand Legacy Code — Regression, Characterization and Approval tests	https://understandlegacycode.com/blog/characterization-tests-or-approval-tests/	2026-08-20	Feathers' stated purpose of characterization testing, and the documented limitation that they preserve current rather than correct behaviour
S18	Test Double — Refactoring Legacy code with tests	https://github.com/testdouble/contributing-tests/wiki/Refactoring-Legacy-code-with-tests	2026-08-20	Write characterization tests against a seam, then delete them after the refactor because redundant coverage is problematic
S19	Wikipedia — Characterization test	https://en.wikipedia.org/wiki/Characterization_test	2026-08-20	The term was coined by Michael Feathers; a test written to document current behaviour and preserve it while the code changes
S20	GitHub Docs — Configuring a publishing source for your GitHub Pages site	https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site	2026-08-20	Branch-based publishing is recommended when no control over the build is needed; a GitHub Actions workflow is recommended for any other build process
S21	Vite — Deploying a Static Site	https://vite.dev/guide/static-deploy	2026-08-20	For a GitHub Pages project site at <code>/<REP0>/</code> , <code>base</code> must be set to <code>'/<REP0>/'</code>

#	Title	URL	Accessed	Supports
S22	Node.js — Test runner	https://nodejs.org/api/test.html	2026-08-20	<code>node:test</code> is documented "Stability: 2 - Stable" and runs via <code>node --test</code> with default file patterns. Page header read as Node.js v26.7.0 documentation on the access date

The walking-skeleton practice referenced in sections 2 and 3 has **no source row**, deliberately. `https://alistair.cockburn.us/walking-skeleton/` returned HTTP 404 on 2026-08-20 and the O'Reilly chapter that reproduces the definition is behind a subscription, so no primary text was fetched. The practice is named as a widely-attested concept and nothing in the argument rests on its exact wording; the refactor-velocity claim it accompanies is carried by [S12] instead. This is recorded rather than papered over with an unverified citation.

9. Quick wins

Five items qualify. Each is independently shippable, none depends on the P0 decision about the diverged `index.html`, none depends on the unresolved Pages publishing source, and none duplicates RP-01 §11. Two are new-files-only and therefore carry zero regression risk for the deployed app; three are single-property CSS or markup changes.

Deliberately excluded, with reasons: removing `beforeunload` (blocked — RP-01 open question 6 is an open decision, and section 5 shows it is a user-visible behavioural change requiring a conversation); deleting the ~35 lines of dead CSS (RP-01 §5 explicitly counsels doing this *during* the move, not before); adopting the diverged `index.html` (blocked by RP-01 open question 1); anything touching the cross-month price logic (blocked by RP-01 open question 3); and adding a Pages Actions workflow (blocked by the publishing-source TBD).

Rank	Quick win	Effort	Impact	Basis of ranking
1	Let the calendar controls wrap and stop clipping the summary at 375 px	XS	High (user)	RP-01 measured <code>#clearMonthBtn</code> extending to <code>x=394</code> in a 375 px viewport and <code>◀</code> clipped at the left edge — two controls unreachable on a phone. Two CSS property changes, no JS, no storage. Highest impact per unit effort in this report
2	Make buttons and form controls inherit the page font family	XS	Medium (user)	RP-01 runtime-verified that the system font stack never reaches buttons, which compute to Arial, because <code>index.html:11</code> sets no font. One declaration, visible on every screen
3	Add a favicon	XS	Low-Medium (user)	RP-01 verified a favicon 404 logging a console error on every load. Removes the app's only console error and gives it a browser-tab identity

Rank	Quick win	Effort	Impact	Basis of ranking
4	Copy the DOM-free domain functions into typed modules with tests	S	High (programme), none (user)	This is phase P1, and it unblocks P2, P3 and P4. Touches no deployed file, so regression risk is structurally zero. Makes the money arithmetic unit-testable for the first time
5	Approval-pin the payment message and the CSV bytes	S	High (programme), none (user)	Pins the only two outputs where a regression is silent and costs the user money or data. Also new-files-only

Ranking basis: impact divided by effort, with ties broken toward items the end user can see. Items 1 to 3 satisfy all four quick-win criteria outright. **Items 4 and 5 are a stated deviation:** they have no direct user-visible effect, so they fail criterion (a) as written, and they are included because they are migration-preparatory work that is shippable today, carries structurally zero regression risk, and is blocked by none of this report's open decisions. Ranked below the user-visible wins for that reason. The report's three user-visible quick wins stand on their own if items 4 and 5 are struck.

PROMPT QW-1: Make the calendar controls usable at 375 px

Context: Repo lesson-planner, single deployed file index.html at the repo root, served by GitHub Pages. The schedule editor's control strip is .calendar-controls, defined at index.html:179 as display:flex with justify-content:center, align-items:center, gap:8px and no flex-wrap, so it defaults to nowrap. RP-01 measured that at a 375 px viewport the row overflows: #clearMonthBtn extends to x=394 and the ◀ button (#prevMonthBtn) is clipped at the left edge, leaving both unreachable without scrolling inside the modal. Separately #calendar-summary at index.html:194-200 sets height: 1em (12 px) while its text needs 14 px, clipping 2 px off the bottom on every render.

Task: Add flex-wrap: wrap to the .calendar-controls rule at index.html:179 so the six controls reflow onto a second line instead of overflowing, and change the fixed height: 1em on #calendar-summary at index.html:199 to a min-height of the same value so the element reserves one line but grows to fit its content.

Constraints: Do not change the markup order of the six controls inside the .calendar-controls div at index.html:318-323, and do not change any element id. Do not change the .calendar or .calendar-dow grid rules. Do not change the calendar summary's text content, its em dash, its font-size, colour or text-align. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. No new dependencies, no build step, no JavaScript change.

Acceptance criteria: At a 375 px viewport width with a group open and the schedule editor showing, the modal's scrollWidth equals its clientWidth, every one of #prevMonthBtn, #monthSelect, #yearInput, #nextMonthBtn, #todayBtn and #clearMonthBtn has a bounding rect fully inside the modal's content box, and all six are clickable without horizontal scrolling. At 1280 px the control row still renders on a single centred line. With one date selected, #calendar-summary has scrollHeight equal to clientHeight and its full text including the em dash is visible.

Verification: Serve the file locally, add a group, open it, click Edit Schedule, then at viewport width 375 assert modal scrollWidth equals clientWidth and read getBoundingClientRect for each of the six control ids, asserting left is at least the modal content left and right is at most the modal content right. Repeat at width 1280 and assert all six rects share one top value. Select one date and assert #calendar-summary scrollHeight equals clientHeight.

PROMPT QW-2: Make buttons and form controls inherit the page font family

Context: Repo lesson-planner, index.html. The body font stack is declared at index.html:9 as system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif. The generic button rule at

index.html:11 sets cursor, border, background, border-radius and padding but no font, and form controls have no font declaration either, so RP-01 runtime-verified that buttons compute to the user-agent default Arial rather than the page stack. Two elements already opt in individually: .payment-message pre at index.html:125 and textarea at index.html:166, both with font-family:inherit. The result today is that every button and every input, select and number field renders in a different typeface from the surrounding text.

Task: Add font-family: inherit to the shared control styling so buttons, inputs, selects and textareas use the page font stack. Prefer adding a single rule covering button, input, select and textarea next to the existing button rule at index.html:11, and remove the now-redundant font-family:inherit from the textarea rule at index.html:166 only if the new rule provably covers it.

Constraints: Change the font FAMILY only. Do not set font-size, font-weight, line-height or the font shorthand, because RP-01's measured target sizes and the 375 px layout assume the current computed text size and a size change would invalidate them. Do not change any padding, border, border-radius, background or colour. Do not change .icon-button's font-size at index.html:132, which sizes the pencil emoji. Do not change the three localStorage key names or the persisted data shape. No new dependencies, no build step, no JavaScript change, no markup change.

Acceptance criteria: The computed font-family of #addGroupBtn, #saveGroupBtn, #monthSelect, #yearInput, #groupNameInput and #templateTextarea equals the computed font-family of body. The computed font-size of #addGroupBtn is unchanged from before the edit. At a 375 px viewport the main screen still has scrollWidth equal to clientWidth, and with the schedule editor open every control in .calendar-controls still has abounding rect inside the modal content box. The pencil icon button still renders at its existing size.

Verification: Serve the file locally and, before and after the edit, record getComputedStyle for the six elements listed above plus body, asserting font-family now matches body and that font-size for #addGroupBtn is identical between the two recordings. Then repeat the 375 px overflow checks from the calendar quick win and confirm they still pass.

PROMPT QW-3: Give the app a favicon so it stops logging a console error on every load

Context: Repo lesson-planner. The deployed site is one file, index.html, with no <link> elements at all – RP-01 runtime-verified zero <link> and zero <script src>. Because no icon is declared, the browser requests /favicon.ico from the domain root rather than from the project path, and RP-01 confirmed on the live site that this 404s and logs a console error on every single page load. It is the app's only console error. The page also has no tab identity beyond its title, Group Lesson Planner, set at index.html:6.

Task: Add an icon to the repository and declare it with a single <link rel="icon"> element in the <head> of index.html, immediately after the <title> at index.html:6, using a path relative to the document so it resolves under the GitHub Pages project subpath rather than at the domain root. State in the commit which file format was chosen. This prompt creates the icon file.

Constraints: Use only a self-contained local file committed to the repository; no CDN, no external URL, no data source outside the repo, preserving the app's zero-external-request property after load. Do not add any other <link>, do not add a web app manifest and do not add a service worker. Do not change the <title> text. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. No new dependencies, no build step, no JavaScript change.

Acceptance criteria: Loading the page produces zero console errors and zero failed network requests. The icon request resolves relative to the document path, not the domain root, so it returns 200 when the site is served from a project subpath. The document contains exactly one <link> element. The page still makes no request to any external host.

Verification: Serve the file locally from a subdirectory that mimics the project subpath, load the page, and assert that the console contains no error entries and that the network log shows every request returning 200 with no request to a third-party host. Confirm the icon appears in the browser tab. Count <link> elements in the served document and assert the count is exactly one.

PROMPT QW-4: Copy the DOM-free domain logic into typed modules with unit tests
Context: Repo lesson-planner. The deployed app is one file, index.html, with a single inline script and no build step; the repository root has no package.json and no tsconfig.json. Statically verified: the whole of App.utils at index.html:1365-1467 and the whole of App.services.groups and App.services.csv at index.html:1214-1361 touch no DOM. Two of those functions read globals rather than parameters: App.utils.generateMonthlyPaymentMessage at index.html:1367-1378 reads the template from localStorage via App.services.storage.getTemplate() at index.html:1207-1209, and App.services.groups.getActiveOverrides at index.html:1242-1245 reads the App.state.isEditing global. A third business rule lives in markup: the currency whitelist exists only as <option> values at index.html:292-293 and is read back out of the DOM at index.html:1001-1002.
Task: Create a new source directory and a new root package.json and tsconfig.json (this prompt creates all of them), and port the following into TypeScript modules as pure functions with explicit parameters and no globals: pad, iso, toMonthKey, startWeekday, escapeCsvValue, parseCsv, normalizeMonthKey, deepClone and formatCurrency from App.utils; cloneOverrides, collectDatesByMonth, ensureOverride, normalizeOverrides and flattenSelectedDates from App.services.groups; serialize and deserialize from App.services.csv; and generateMonthlyPaymentMessage rewritten to take the template string and the overrides object as parameters instead of reading localStorage and App.state. Encode the currency whitelist as a TypeScript union type of the two values currently present as <option> values, rather than reading the DOM. Add unit tests covering each exported function, including the RP-01 inputs known to misbehave: a one-digit year producing a date key whose year part is shorter than four characters, a currency value that is not exactly three ASCII letters, and a CSV containing a stray quote that happens to balance. Add an npm test script that runs the Node built-in test runner.
Constraints: Do NOT modify index.html at all – this is a deliberate temporary duplication, and the deployed file must keep its zero-dependency, zero-build, single-file property until a later cutover. Do not introduce a test framework dependency; use node:test and node:assert. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape – the ported CSV and override functions must produce byte-identical output to the current implementation for the same input. Do not port App.utils.formatDate, which RP-01 verified is never called. Do not port App.render, App.handlers or App.services.storage. Do not use the any type. Tests must not embed the default payment template from index.html:382-400, which contains a real person's name, bank identifiers and tax identifier – use a neutral template with the three tokens only.
Acceptance criteria: The npm test script exists in the new root package.json, runs, and every test passes. Every function named in the Task has a typed export and at least one test. serialize applied to a group fixture produces a string whose first row is the six quoted column headers in the existing order, whose rows end with CRLF, and which contains no UTF-8 byte order mark, matching the current return value of App.services.csv.serialize exactly; note that a byte order mark, if it is added later per RP-01's own separate quick win, is applied at the Blob layer in saveToCsv and not inside serialize, so it is out of scope here. deserialize applied to that same string returns a groups array deep-equal to the input fixture. normalizeMonthKey throws for a month value of 13 and for a value with a one-digit year. formatCurrency throws for a currency value that is not exactly three ASCII letters, and the currency union type does not permit such a value. No file outside the new directories and the two new root config files is modified; git status shows index.html unchanged. A grep over every new file returns zero matches for the fixed strings IBAN and MFO case-insensitively and zero matches for any digit run of 8 or more.
Verification: Run the new npm test script and confirm all tests pass. Run git status and confirm index.html, AGENTS.md and LICENSE are unmodified. Run the two greps named in the acceptance criteria and confirm both return nothing.

PROMPT QW-5: Approval-pin the generated payment message and the CSV export bytes
Context: Repo lesson-planner. RP-01 established that the two outputs whose regression is

silent and costly are the substituted payment message, measured at 446 characters for the default template, and the CSV export bytes, whose format is a header row of six quoted columns, CRLF row endings, every field quoted, dates space-delimited inside one quoted field, and no UTF-8 byte order mark. Both are produced by functions that are pure once their inputs are parameters: the message by `App.utils.generateMonthlyPaymentMessage` at `index.html:1367-1378` and the CSV by `App.services.csv.serialize` at `index.html:1251-1282`. This prompt is independently shippable in either of two ways: run it after the typed-module port described in quick win 4 and target those modules, or run it standalone by copying just those two functions into a test-local module first. It creates new test and fixture files and modifies no existing file.

Task: Add approval tests, also called golden-master tests, that render each of the two outputs for a small fixed fixture and compare the whole output against a committed approved file rather than asserting field by field. Cover: the substituted payment message for a group with one lesson and for a group with four lessons; the serialized CSV for a group with no months, a group with two months, and a group whose name contains a comma, a double quote and Cyrillic characters. Alongside each approved file, record in the test itself an explicit label stating whether that output is contract, meaning it must never change, or defect, meaning it is expected to change when the corresponding RP-01 defect is fixed. Label the singular-count message and the CSV outputs contract. Label nothing defect unless it corresponds to a numbered RP-01 defect, and name that defect in the label.

Constraints: The fixture template must be a neutral placeholder containing only the three tokens for month, lessons and total plus surrounding neutral text. It must NOT be the default template at `index.html:382-400`, which contains a real person's full name, bank IBAN and tax identification number, and none of those values may appear in any test, fixture, approved file or comment. Do not modify `index.html`. Do not modify the ported functions to make them easier to test. Do not introduce an approval-testing library; a committed expected-output file plus a whole-string comparison is sufficient. Currency formatting must stay pinned to the existing en-US locale so approved files are stable across machines. Any test that touches the current date must use a fixed date, not the system clock.

Acceptance criteria: Every approved file is committed and every approval test passes on a second run with no file rewritten. Each approval test carries a contract-or-defect label naming the RP-01 defect where applicable. Changing the price in a fixture by one unit makes exactly the affected approval test fail, proving the pins are load-bearing rather than vacuous. Reordering the CSV columns in the ported serializer makes the CSV approval tests fail. A grep over every new file returns zero matches for the fixed strings IBAN and MFO case-insensitively and zero matches for any digit run of 8 or more.

Verification: Run the test script twice and confirm all tests pass both times and that git status shows no approved file modified by the run. Temporarily change one fixture price by one unit, re-run, confirm exactly the expected tests fail, then revert. Run the greps in the acceptance criteria and confirm both return nothing.